ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет об исследовательском проекте

на тему «Компиляторная оптимизация вычислительных графов для локального выполнения больших языковых моделей»

Выполнил студент:

группы #БПМИ236, 3 курса Горохов Дмитрий Александрович

Принял руководитель проекта:

Кулагин Иван Иванович, к. т. н.
Научный сотрудник
Федеральное государственное бюджетное учреждение науки
Институт системного программирования им. В.П. Иванникова
Российской академии наук (ИСП РАН)

Москва 2025

Содержание

1 Аннотация **3**
2 Ключевые слова **3**
3 Введение **3**
4 Цели и задачи **5**
5 Анализ современных средств хранения, оптимизации и инференса моделей машинного обучения **6**
5.1 Исследование особенностей низкоуровневой библиотеки тензорных вычислений GGML 6
5.1.1 Программные компоненты проекта 6
5.2 Описание формата хранения и сериализации моделей GGUF 7
5.3 Рассмотрение архитектуры системы инференса больших языковых моделей Llama.cpp . . 8
5.3.1 Интерфейс 8
5.3.2 Квантование 9
5.3.2.1 Простейшие техники квантования 10
5.3.2.2 K-кванты 10
5.3.2.3 Importance matrix 10
5.4 Обзор возможностей фреймворка построения компиляторов MLIR 10
5.4.1 Введение 10
5.4.2 Диалекты 12
5.4.2.1 Диалект Tensor 12
5.4.2.2 Диалект SCF 12
5.4.2.3 Диалект Affine 13
5.4.2.4 Диалект Linalg 13
5.4.3 Преобразования 14
5.4.3.1 Понижающие 14
5.4.3.2 Оптимизирующие 15
6 Разрабатываемый компилятор **15**
6.1 Архитектура 15
6.2 Тестирование 17
6.3 Оптимизирующая трансформация Tiling 20
7 Заключение **23**
Библиография **24**

1 Аннотация

Развитие моделей машинного обучения и увеличение их размеров требует значительных вычислительных ресурсов для их эффективного выполнения. По этой причине в мире повысился интерес к разработке специализированных процессоров для ускорения выполнения нейронных сетей. Эффективное использование всех доступных возможностей целевого аппаратного обеспечения невозможно без развитого стека программного обеспечения. По этой причине исследования в области построения оптимизирующих компиляторов нейронных сетей являются критически важными и актуальными. На сегодняшний день формат GGUF и библиотека llama.cpp стали промышленным стандартом для эффективного выполнения больших языковых моделей на локальных устройствах, в особенности на устройствах с ограниченными вычислительными ресурсами. К числу последних можно отнести встраиваемые системы, устройства, работающие под управлением энергоэффективных процессоров. Однако эта библиотека не выполняет оптимизирующих трансформаций вычислительного графа, а только выполняет его при помощи вручную созданных kernel-функций. В данной работе предлагается разработать компилятор на базе инфраструктуры MLIR. Разрабатываемый компилятор будет выполнять трансляцию операций вычислительного графа проекта GGML (подпроект llama.cpp) в операции линейной алгебры, реализованные в виде MLIR-диалекта linalg. Это позволит проводить аппаратно-независимые и архитектурно-ориентированные оптимизации операций линейной алгебры. В рамках проекта также планируется поддержать некоторые алгоритмы квантования, используемые в инфраструктуре llama.cpp.

2 Ключевые слова

Компилятор нейронных сетей, MLIR, захват вычислительного графа, LLM, нейронные сети, Llama.cpp, GGML, оптимизирующий компилятор, тензорные вычисления

3 Введение

Эффективное исполнение (инференс) моделей машинного обучения (таких как большие языковые модели, генеративные предобученные трансформеры, мультимодальные трансформеры) требует развитого стека программного обеспечения. Большинство фреймворков ориентированы на серверы: системы с большим объемом вычислительных ресурсов и специализированными видеоускорителями (GPU). В настоящее время наблюдается повышенный интерес к разработке специализированных процессоров для ускорения выполнения нейронных сетей и тензорных вычислений, таких как Google TPU, Apple Neural Engine, Qualcomm Hexagon, Groq LPU, Intel Gaudi

и других, в том числе для edge-устройств, таких как Axon NPU, Coral NPU, Ethos-U65 NPU и так далее, а также к запуску моделей в «домашних» условиях: на CPU с ограниченным количеством оперативной памяти.

В настоящий момент де-факто лидером выполнения больших языковых моделей на такого рода устройствах является библиотека llama.cpp и используемый в ней для инференса вычислительного графа фреймворк для тензорных и матричных вычислений GGML. Поддерживаемые ими бэкенды включают в себя Metal, MUSA, CUDA, HIP, ZenDNN, CANN, IBM zDNN и прочие. В этих библиотеках реализованы многие оптимизации, причем на разных уровнях. На уровне llama.cpp реализованы различные стратегии квантования (перевод из одного типа данных в тип данных с меньшей шириной, сопровождается потерей точности) исходных тензоров для экономии памяти, оптимизированные функции вычисления RoPE, Flash Attention, оптимизированный key-value cache. На уровне GGML реализованы эффективная работа с памятью (memory pooling, различные zero-copy стратегии), оптимизации планировщика. Однако в силу того, что GGML создает вычислительный граф динамически, а при исполнении обходит его, вызывая kernel-функции (вычислительные функции для конкретной архитектуры, например matmul, оптимизация которых полностью лежит на разрабатывавшем их программисте), множество возможных оптимизаций не применяются.

Это приводит к необходимости создания оптимизирующего компилятора нейронных сетей, который использует в своей работе статический вычислительный граф и при этом учитывает особенности различных целевых архитектур, на выходе которого получается бинарный файл для требуемой архитектуры. Это позволит применять как многие архитектурно-независимые, так и архитектурно-зависимые оптимизации на уровне всего графа. Существуют похожие решения, например, executorch для torch, аналогично захватывает динамический вычислительный граф; TensorFlow [1] изначально предполагает работу со статическим графом. При этом ни одно из решений не позволяет запускать модели, сохраненные в формате GGUF, и пользоваться уже реализованными в llama.cpp оптимизациями, лежащими выше уровня вычислительного графа, которые на практике оказались крайне полезны, особенно для инференса на edge-устройствах.

Таким образом, в рамках данной курсовой работы был реализован прототип такого компилятора [2]. На вход он принимает модель в формате GGUF, а на выходе получается бинарный файл, оптимизированный под конкретную целевую архитектуру. База такого компилятора — MLIR: проект внутри инфраструктуры LLVM, позволяющий работать с IR (промежуточным

представлением) на разных уровнях абстракции. Изначально граф задается в терминах высокоуровневых тензорных операций. С помощью MLIR данное представление можно понизить до уровня аффинных циклов, потом до уровня графа потока управления и в конце сгенерировать код для требуемой целевой архитектуры. При этом на каждом этапе возможно производить оптимизирующие проходы, возможные только на этом уровне абстракции.

4 Цели и задачи

Целью данной курсовой работы является разработка компилятора нейронных сетей, представленных в формате GGUF. В рамках данной цели были поставлены следующие задачи:

1. Исследовать функциональные возможности и структуру фреймворка Llama.cpp и используемой в ней библиотеки GGML, устройство формата файла GGUF.
   - Проанализировать процесс создания вычислительного графа, формат его хранения и способ обхода.
   - Определить способ захвата вычислительного графа.
2. Исследовать возможности инфраструктуры MLIR.
   - Изучить внутренние сущности: диалекты, типы, проходы; способы их создания на декларативном языке или на `C++`
   - Рассмотреть реализованные оптимизации, их применимость к вычислительному графу.
3. Реализовать компилятор нейронных сетей, представленных в виде вычислительного графа библиотеки GGML. Для этого необходимо:
   - Извлечь веса константных тензоров вычислительного графа нейронной сети, хранящихся в файле формата GGUF.
   - Реализовать захват вычислительного графа GGML.
   - Реализовать собственный диалект MLIR GGML, содержащий прототипный набор операции линейной алгебры.
   - Разработать транслятор операций диалекта GGML в набор операций диалекта LLVM, реализующий прогрессивное понижение.
4. Разработать тестовый набор для оценки корректности и эффективности компилятора нейронных сетей и оптимизирующих трансформаций.
5. Исследовать эффективность разработанного компилятора и применяемых оптимизирующих трансформаций.

5 Анализ современных средств хранения, оптимизации и инференса моделей машинного обучения

5.1 Исследование особенностей низкоуровневой библиотеки тензорных вычислений GGML

GGML [3] — библиотека для тензорных вычислений для машинного обучения. Написана на C/C++, причем ядро библиотеки написано на чистом C для простоты создания байндингов для других языков (Python, Go, Rust, Node.js) и производительности. Не все возможности C++ используются, так как версия стандарта — C++11, нет исключений и информации о типах во время исполнения (Runtime Type Information). Похожа на другие библиотеки для машинного обучения, такие как PyTorch и TensorFlow. Обрела популярность во многом благодаря проекту llama.cpp, однако используется и в других проектах, например, ollama [4]. Основные причины, по которым люди выбирают GGML:

- Минимализм.
- Простота компиляции.
- Легковесность.
- Хорошая совместимость.
- Поддержка квантованных вычислений.
- Эффективность использования памяти.

Основные минусы проекта включают в себя:

- Не все тензорные операции поддержаны на всех бэкендах.
- Разработка с использованием GGML менее прямолинейна, чем с использованием аналогичных библиотек, и требует знаний в низкоуровневом программировании.
- Проект активно разрабатывается и подвержен изменениям, нарушающим совместимость.

5.1.1 Программные компоненты проекта

Главным строительным блоком библиотеки является структура `ggml_tensor`

```
struct ggml_tensor {
    enum ggml_type type; // тип тензора
    struct ggml_backend_buffer * buffer; // память тензора
    int64_t ne[GGML_MAX_DIMS]; // форма тензора
    enum ggml_op op; // операция, результатом которой является тензор
```

```
    ...
    struct ggml_tensor * src[GGML_MAX_SRC]; // операнды этой операции
    char name[GGML_MAX_NAME]; // название тензора
};
```

Константные тензоры (не являющиеся результатами каких-либо операций) хранят в переменной `op` специальное значение `GGML_OP_NONE`.

Благодаря такому определению тензора вычислительный граф задается как их список

```
struct ggml_cgraph {
    ...
    struct ggml_tensor ** nodes;
    ...
};
```

Стоит отметить, что библиотека также поддерживает подсчет частных производных и различные оптимизаторы для обучения моделей. Данная функциональность не используется в Llama.cpp и не будет использована в рамках курсовой работы.

5.2 Описание формата хранения и сериализации моделей GGUF

GGUF [5] — файловый формат для сохранения моделей для инференса с помощью GGML. Является бинарным форматом, спроектированным для быстрой загрузки и сохранения моделей, а также для простоты парсинга. Модели обычно разрабатываются на PyTorch или другом фреймворке, а в дальнейшем конвертируются в GGUF. Формат был разработан для поддержки следующей функциональности:

- Вся информация для модели хранится в одном файле: это упрощает распространение и загрузку моделей.
- Расширяемость: новая функциональность может быть добавлена в GGUF, при этом совместимость с существующими моделями должна сохраняться.
- Поддержка mmap: модели могут загружаться, используя mmap, для ускорения загрузки и сохранения.
- Простота использования: модели могут быть легко загружены и сохранены, используя небольшое количество кода, без необходимости в дополнительных библиотеках, на любом языке программирования.

На рисунке 1 показана структура GGUF-файла.

Рис. 1. Структура GGUF-файла

5.3 Рассмотрение архитектуры системы инференса больших языковых моделей Llama.cpp

Llama.cpp [6] — библиотека для запуска LLM. Среди целей проекта следует отметить простоту использования и поддержку множества аппаратных архитектур. Разработана на C/C++ с минимальным числом зависимостей. Использует GGML в качестве фреймворка машинного обучения для выполнения тензорных операций. На момент написания работы поддерживает 123 различные архитектуры моделей машинного обучения, среди них: LLaMA, BERT, архитектуры Deepseek, архитектуры Qwen, GPT-2, Grok-1, SmolLM и GigaChat.

5.3.1 Интерфейс

Опишем классический пример использования Llama.cpp. Пользователь обладает моделью в формате GGUF и промптом — текстом, продолжение которого требуется сгенерировать. Взаимодействие с библиотекой для него будет выглядеть так:

1. Загрузить модель в память. Для этого нужно передать путь до нее в функцию `llama_model_load_from_file`. Функция вернет объект типа `llama_model`. Рассмотрим этот тип подробнее

```
struct llama_model {
    ...
    llm_arch arch; // архитектура модели
    ...
    llama_vocab vocab; // тип словаря
    ...
    struct ggml_tensor * tok_embd; // стандартные тензоры (порядка 20 штук)
    ...
    std::vector<llama_layer> layers; // слои
    ...
    ggml_cgraph * build_graph(const llm_graph_params & params) const; // функция,
возвращающая вычислительный граф
    ...
};
```

2. Для перевода текста в токены и обратно необходим словарь. Его можно получить функцией `llama_model_get_vocab`.
3. Далее происходит токенизация промпта функцией `llama_tokenize`.
4. Далее в цикле генерируются следующие токены. Для этого используется функция `llama_decode`. Она и занимается вычислением графа. Для вывода пользователю токены необходимо переводить обратно в их текстовое представление. Цикл заканчивается или через определенное число итераций, или когда модель вернет специальный токен конца текста.

5.3.2 Квантование

Количество параметров в моделях машинного обучения, а особенно в больших языковых моделях, постоянно растет. В настоящее время распространены модели с десятками и даже сотнями миллиардов параметров. В свою очередь, объемы оперативной памяти (RAM) и памяти ускорителей (VRAM) увеличиваются недостаточно быстро. Для решения данной проблемы было предложено квантование: сжатие параметров, влекущее за собой потерю точности в обмен на уменьшение требуемой памяти. В ходе экспериментов выяснилось, что значительное понижение используемой памяти влечет намного менее значительное понижение качества модели. Особенно необходимо квантование в условиях крайней ограниченности доступной памяти.

Описание стандартных типов: обычно модели используют или числа одинарной точности (float32, F32), или числа половинной точности (float16, F16). Целые числа любой ширины: int_k (I_k) может принимать 2^k различных значений.

5.3.2.1 Простейшие техники квантования

Рассматривается блок чисел (размером 32 в GGML). Симметричное квантование (type-0 в GGML). Дан блок чисел w_i. Хотим для него хранить float d (коэффициент масштабирования) и массив квантов int_k q_i, так что $w_i = d \cdot q_i$.

Пусть $a = \max(|w_i|)$. Получается мы хотим сжать отрезок $[-a, a]$ в отрезок $\left[-2^{k-1}, 2^{k-1}\right]$. Таким образом в качестве d следует взять $\frac{a}{2^{k-1}}$. Кванты вычисляются по формуле $q_i = \left\lfloor \frac{w_i}{d} \right\rceil$ Аффинное квантование (type-1 в GGML) Дан блок чисел w_i. Хотим для него хранить два float: d (коэффициент масштабирования) и m (минимум), а также массив квантов int_k q_i, так что $w_i = m + d \cdot q_i$.

Пусть $m = \min(w_i)$, а $a = \max(w_i)$. Получается мы хотим сжать отрезок $[m, a]$ в отрезок $\left[0, 2^k\right]$. Таким образом в качестве d следует взять $\frac{a-m}{2^k}$. Кванты вычисляются по формуле $q_i = \left\lfloor \frac{w_i - m}{d} \right\rceil$

5.3.2.2 K-кванты

Используются супер-блоки. Каждый супер-блок содержит 16 блоков, каждый из которых содержит 16 чисел. В зависимости от метода для каждого блока выполняется или аффинное, или симметричное квантование, после чего их коэффициенты масштабирования (d) и минимумы (m) квантуются.

5.3.2.3 Importance matrix

Идея заключается в том, что какие-то параметры важнее других. Подавая большое количество различных входных данных в модель, можно вычислить Importance matrix — матрицу важности каждого скаляра. Используя эту информацию, можно сжимать тензоры более эффективно, получать лучшее качество за меньшую память.

5.4 Обзор возможностей фреймворка построения компиляторов MLIR

5.4.1 Введение

MLIR [7] (Multi-Level Intermediate Representation) — проект компиляторной инфраструктуры в экосистеме LLVM [8], возникший как попытка его авторов решить проблему того, что совре-

менные им фреймворки машинного обучения состоят из множества различных компиляторов, графовых технологий и систем исполнения, не имеющих общей платформы. Пример программы `a + b` на MLIR:

```
module {
  func.func @main() -> i32 {
    %a = arith.constant 10 : i32
    %b = arith.constant 20 : i32
    %sum = arith.addi %a, %b : i32
    return %sum : i32
  }
}
```

Перечислим некоторые из основных особенностей:

- Система построена на минимальном количестве фундаментальных концептов, что позволяет большинству промежуточного представления быть полностью кастомизированным. Несколько абстракций (встречающиеся в большинстве IR): типы, операции и атрибуты должны быть использованы, чтобы выражать все остальное. Настраиваемость позволяет системе адаптироваться к изменяющимся требованиям и с большей вероятностью быть применимой к будущим задачам. В итоге MLIR позволяет выражать разнообразный набор абстракций, включающий в себя графы машинного обучения, математические абстракции, такие как полиэдры, графы потока управления и промежуточные представления уровня инструкций, такие как LLVM IR, несмотря на отсутствие концепций из этих абстракций в системе.
- SSA-форма [9] широко используется в промежуточных представлениях компиляторов. Она предоставляет множество преимуществ, таких как упрощение анализа потока данных, широко принята в компиляторном коммьюнити и используется в основных фреймворках. В то время как многие IR используют плоские, линейные графы потока управления, представление абстракций более высокого уровня заставляет добавить *вложенные регионы* как концепт IR. Это позволяет пользоваться абстракциями более высокого уровня, ускорять компиляцию, использовать SIMD-параллелизм.
- Система поддерживает постепенное понижение уровня репрезентации с более высокого до самого низкого, причем понижение выполняется маленькими шажками в условиях нескольких уровней абстракции. Необходимость в нескольких уровнях абстракции возникает из-за разнообразия платформ и моделей программирования, которые должен поддерживать проект универсальной компиляторной инфраструктуры.

MLIR — модульный и легко расширяемый. Для этого он использует концепт диалектов. Репозиторий MLIR содержит множество диалектов: affine, arith, cf, complex, linalg, llvm, math, scf, tensor и другие. Главное же — возможность пользователей создавать свои диалекты. Так, существуют диалекты, разработанные независимыми от разработчиков MLIR командами: stablehlo, iree, onnx, torch. В рамках данной курсовой работы также будет разработан отдельный диалект GGML. Диалекты содержат операции: `arith.addi`, `linalg.matmul`, `scf.for`; типы: `i32`, `tensor<4x4xf32>`, `llvm.void`; и преобразования: они могут быть как оптимизирующими: `remove-dead-values`, `affine-loop-unroll`, так и переводящими из одних диалектов в другие (понижающими): `lower-affine`, `convert-scf-to-cf`, `convert-cf-to-llvm`. Рассмотрим некоторые диалекты подробнее.

5.4.2 Диалекты

5.4.2.1 Диалект Tensor

Тип `tensor`, например, `tensor<4x4xf32>`, `tensor<1x3xi32>`, является абстракцией для N-мерного тензора с фиксированным типом. Данный тип специально абстрагирован от памяти, лежит на более высоком уровне абстракции: для низкоуровневой работы, в том числе с многомерными буферами, существует тип `memref`. Тензоры иммутабельны; таким образом, при использовании их в качестве аргумента всегда происходит копия. Однако в этом нет ничего страшного: после понижения в диалекты более низкого уровня создание лишних копий может быть удалено оптимизирующими проходами. Обратим внимание, что сам тип не является частью диалекта `tensor`, а лежит в `Builtin`-диалекте.

Данный же диалект содержит операции создания и манипуляции объектами данного типа. Необходимые нам операции:

- `tensor.empty` — создает пустой тензор заданной формы.
- `tensor.extract` — возвращает элемент тензора с заданным индексом.
- `tensor.extract_slice` — возвращает подтензор с заданными координатами и размером.
- `tensor.insert_slice` — вставляет подтензор на заданное место.

5.4.2.2 Диалект SCF

Диалект Structured Control Flow содержит операции для создания структурированных конструкций потока управления, таких как `for` и `if`. Под структурированностью подразумевается наличие структуры, в отличие от `goto` или `assert`. Рассмотрим пример `scf.for` для суммирования чисел от 1 до 10.

```
%zero = arith.constant 0 : i32
%one = arith.constant 1 : i32
%eleven = arith.constant 11 : i32

%result = scf.for %i = %one to %eleven step %one iter_args(%sum = %zero) -> (i32) {
  %new_sum = arith.addi %sum, %i : i32
  scf.yield %new_sum : i32
}
```

Операция принимает индуктивную переменную, нижнюю и верхнюю границы и шаг. Для поддержания SSA-формы переменные, которые будут обновляться в цикле, должны быть перечислены в атрибуте `iter_args`, а их обновленные значения должны быть перечислены как аргументы операции `scf.yield` в конце цикла. Это упрощает как анализ потока данных, так и понижающие преобразования.

5.4.2.3 Диалект Affine

Этот диалект позволяет задавать гнезда аффинных циклов (границы вложенных циклов, индексы для обращения к памяти должны описываться аффинными преобразованиями), которые в дальнейшем можно оптимизировать, используя техники полиэдральной модели. Пример аффинного преобразования: `affine_map<(d0, d1, d2) -> (d1, d0, 5 + d0 + d1 * 2 + d2 * 16)>`

5.4.2.4 Диалект Linalg

Данный диалект позволяет декларативно задавать математические операции над тензорами. Это дает максимум возможностей для их оптимизации: еще больше, чем affine. Для описания операций используется `linalg.generic`, все именованные операции, такие как `linalg.add`, `linalg.matmul`, `linalg.conv_1d`, на самом деле раскрываются в `linalg.generic`. Рассмотрим пример умножения двух матриц

```
func.func @matmul(
    %A: tensor<8x10xf32>,
    %B: tensor<10x16xf32>,
    %C: tensor<8x16xf32>
) {
    linalg.generic {
        indexing_maps = [
            affine_map<(i, j, k) -> (i, k)>,
            affine_map<(i, j, k) -> (k, j)>,
            affine_map<(i, j, k) -> (i, j)>
```

```
        ],
        iterator_types = [
            "parallel",
            "parallel",
            "reduction"
        ]
    } ins(%A, %B : tensor<8x10xf32>, tensor<10x16xf32>) outs(%C : tensor<8x16xf32>) {
        ^bb0(%lhs_one: f32, %rhs_one: f32, %init_one: f32):
            %tmp0 = arith.mulf %lhs_one, %rhs_one : f32
            %tmp1 = arith.addf %init_one, %tmp0 : f32
            linalg.yield %tmp1 : f32
    }
    return
}
```

`indexing_maps` задает, по каким индексам происходят обращения к входным и выходным тензорам. `iterator_types` задает для каждого цикла его тип, `ins` задает входные тензоры и их типы, `outs` задает выходные тензоры и их типы, `^bb0` является блоком кода, в который передаются значения скаляров по соответствующим координатам входных тензоров и скаляр с соответствующими координатами выходного тензора, хранящий накопленное значение. Внутри блока производятся математические преобразования, и в конце операцией `linalg.yield` в выходном тензоре на соответствующих координатах устанавливается переданное в нее значение. В данном примере будет три цикла по переменным `i`, `j` и `k`. При этом на конкретной итерации цикла `(i, j, k)` в `^bb0` будут переданы `A[i, k]`, `B[k, j]` и `C[i, j]`, и будет выполнено `C[i, j] += A[i, k] * B[k, j]`.

5.4.3 Преобразования

5.4.3.1 Понижающие

- `one-shot-bufferize`: переводит абстрактные тензоры `tensor` в физические области памяти `memref`, вставляет операции аллокации.
- `convert-linalg-to-affine-loops`: конвертирует операции диалекта `linalg` в гнезда аффинных циклов.
- `convert-linalg-to-loops`: конвертирует операции диалекта `linalg` в циклы диалекта `SCF`.
- `lower-affine`: понижает аффинные циклы диалекта `affine` в циклы диалекта `SCF`.
- `convert-scf-to-cf`: переводит структурированные операции управления в неструктурированные: из `SCF` в `CF`.

- `convert-to-llvm`: понижает различные диалекты в `LLVM`-диалект. Обратим внимание, что `LLVM`-диалект MLIR и LLVM IR — не одно и то же.
- и другие.

5.4.3.2 Оптимизирующие

- `affine-loop-tile`: разбиение цикла на блоки. Данная оптимизация будет детально проанализирована в главе 6.3.
- `affine-loop-fusion`: слияние аффинных циклов.
- `cse`: удаление общих подвыражений.
- `canonicalize`: приведение промежуточного представления к упрощенному (каноническому) виду.
- `affine-loop-unroll`: развертывание циклов.
- и прочие.

6 Разрабатываемый компилятор

6.1 Архитектура

Архитектурные компоненты будут рассматриваться по ходу исполнения программы. Схема функциональной структуры компилятора представлена на рисунках 2 и 3: время идет слева направо, цветными прямоугольниками обозначены модули компилятора, светлым прямоугольникам соответствуют функции, на стрелках записаны аргументы и возвращаемые значения.

Разрабатываемый компилятор принимает на вход файл с описанием модели в формате GGUF.

Изначально создается MLIR-модуль, содержащий сигнатуру функции `@compute_graph_forward`. Эта функция будет постепенно наполняться в процессе захвата вычислительного графа, за это отвечает класс `MLIRGen`.

Файл модели `model.gguf` передается функции `llama_load_model` библиотеки Llama.cpp. Данная функция загружает необходимые тензоры в память и возвращает вычислительный граф. Функции `llama_decode`, которая непосредственно занимается вычислением графа, можно передать `callback`, который будет вызываться при обработке очередного тензора. Благодаря этой функциональности и удается захватить вычислительный граф.

Функции `callback` передается следующий тензор, который будет вычислен в формате, описанном в главе 5.1.1. Функция `appendOp` занимается тем, что добавляет вычисление данного тензора в MLIR-функцию `@compute_graph_forward`.

У каждого тензора в GGML-графе есть название. Однако эти названия могут совпадать у разных тензоров — то есть эти названия не соответствуют SSA-форме, требуемой в MLIR. Для решения этой проблемы класс `MLIRGen` поддерживает живые переменные: для каждого названия тензора хранится имя его последней версии в MLIR-программе.

Функция `appendOp` состоит из двух шагов. На первом шаге она обрабатывает операнды. Здесь могут быть две ситуации: операнд — тензор, вычисленный или загруженный ранее, или тензор, еще не встречавшийся. Последняя ситуация означает, что операнд-тензор — константа, хранящаяся в GGUF-файле. При возникновении такой ситуации загрузка этого тензора добавляется в MLIR-программу. По результатам первого шага операндам GGML-операции сопоставляются SSA-переменные тензоров MLIR-функции. Второй шаг: добавление операции.

Для разрабатываемого компилятора был реализован прототип MLIR-диалекта MLIRGGML, полностью повторяющий семантику GGML. Описание операций происходит в tablegen-файлах. Например, для операции `GGML_OP_ADD` в диалекте создана операция `ggml.ADD` следующим образом

```
def AddOp : GGML_Op<"ADD"> {
    let arguments = (ins F32Tensor:$lhs, F32Tensor:$rhs);
    let results = (outs F32Tensor);

    let builders = [
        OpBuilder<(ins "Value":$lhs, "Value":$rhs), [{
            build($_builder, $_state, lhs.getType(), lhs, rhs);
        }]>
    ];
}
```

Таким образом, добавление операции сводится к добавлению в конец MLIR-функции `@compute_graph_forward` операции MLIRGGML-диалекта, соответствующей операции GGML.

По завершении работы функции `llama_decode` MLIR-функция `@compute_graph_forward` содержит весь вычислительный граф, описанный в терминах диалекта MLIRGGML. Далее идет процесс перевода в низлежащие диалекты: сначала в Linalg, Tensor и SCF, а затем и в LLVM. Для

первого в диалекте MLIRGGML реализован `LoweringPass`. Для каждой операции MLIRGGML описано, как заменить ее на операции Linalg, Tensor и SCF, сохраняя семантику. Например, для операции `ggml.ADD`

```
struct AddOpLowering : public OpConversionPattern<ggml::AddOp> {
    using OpConversionPattern<ggml::AddOp>::OpConversionPattern;
    using OpAdaptor = typename OpConversionPattern<ggml::AddOp>::OpAdaptor;

    LogicalResult matchAndRewrite(ggml::AddOp op, OpAdaptor adaptor,
                                  ConversionPatternRewriter &rewriter) const final {
        auto initTensor =
            tensor::EmptyOp::create(rewriter, rewriter.getUnknownLoc(),
op.getLhs().getType().getShape(),
                                    op.getLhs().getType().getElementType());
        auto addOp = linalg::AddOp::create(rewriter, rewriter.getUnknownLoc(),
{op.getLhs(), op.getRhs()},
                                           {initTensor});

        rewriter.replaceOp(op, addOp);
        return success();
    }
};
```

Для понижения в LLVM-диалект MLIR используются стандартные понижающие преобразования MLIR, описанные ранее в главе 5.4.3.

По итогу работы компилятор возвращает скомпилированный вычислительный граф модели в одном из следующих форматов MLIR: GGML, Linalg или LLVM.

6.2 Тестирование

Для разработки и тестирования были внесены изменения в библиотеку Llama.cpp: была добавлена тестовая модель, описан ее вычислительный граф, требуемые ей тензоры. Для этой модели были созданы тестовые GGUF-файлы. Это предоставило возможность тестировать отдельные операции, корректность их понижения и дальнейшей кодогенерации, а не целые модели, имеющие сложные архитектуры.

При указании соответствующего флага `DEBUG` компилятор проверяет корректность скомпилированной программы: пользуется функционалом MLIR для запуска JIT, получая результат

вычисления графа с помощью программы на MLIR, и сравнивает его с результатом, полученным с помощью библиотеки GGML в процессе захвата графа. Был добавлен флаг `nOperations`, чтобы сравнивать результат не всего графа, а лишь его префикса из n шагов.

Также в качестве отдельного режима (вместо генерации MLIR) была добавлена возможность проверять корректность заданного MLIR-файла, содержащего вычислительный граф конкретной модели. Это позволяет вносить правки вручную или применять оптимизирующие преобразования и быть уверенным в корректности сделанных изменений.

Рис. 2. Функциональная структура компилятора (часть 1)

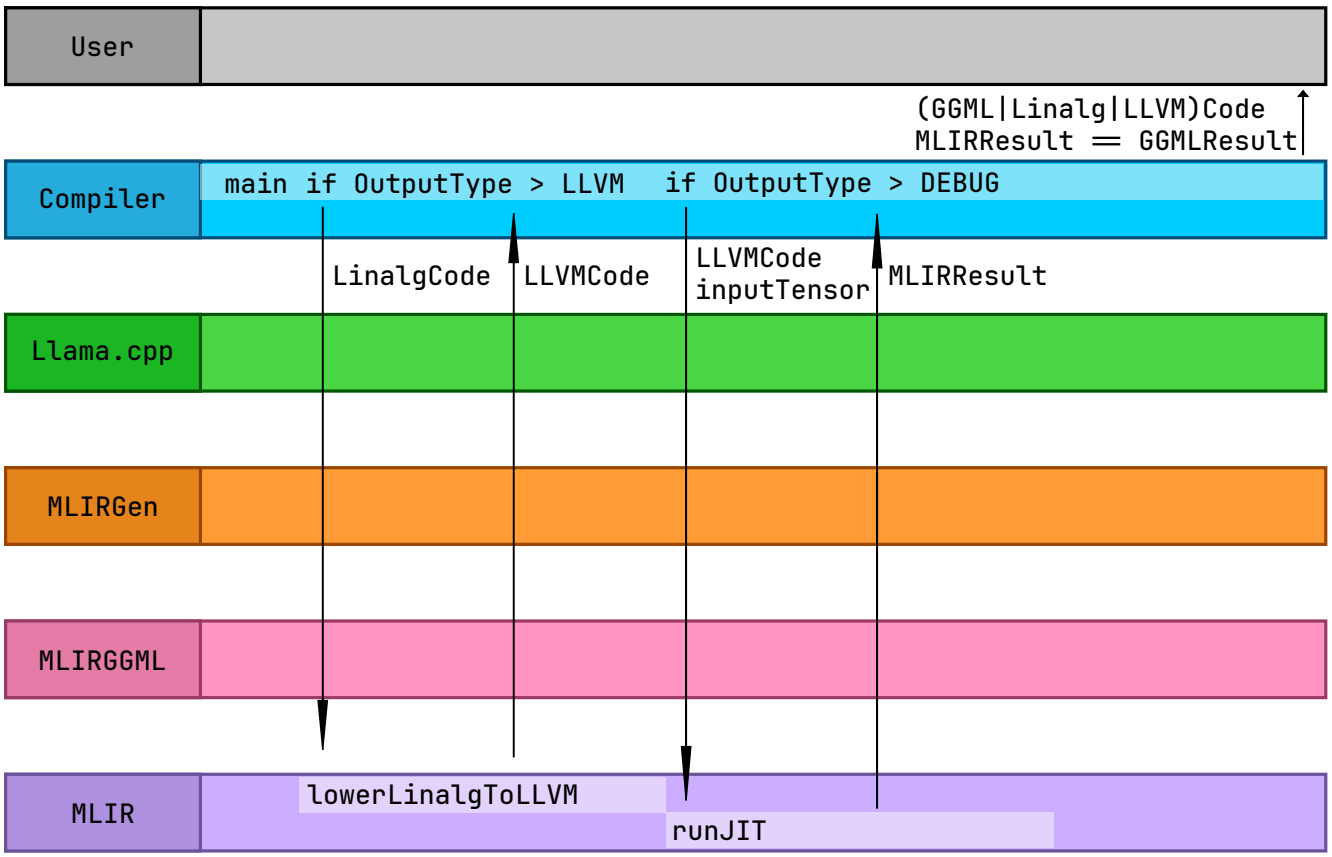

Рис. 3. Функциональная структура компилятора (часть 2)

6.3 Оптимизирующая трансформация Tiling

Разбиение цикла на блоки (loop tiling) [10] является важнейшей техникой оптимизации, улучшающей паттерны обращения к памяти и утилизации кэша. В современных компьютерных архитектурах именно обращения к памяти, а не вычисления, часто являются «бутылочным горлышком». Если обрабатывать большие тензоры линейно, возникает необходимость часто обращаться к основной памяти, а не к кэшу.

Tiling 4 разбивает большие вычисления на меньшие блоки, которые лучше помещаются в кэш. Например, для умножения матриц вычисление полного скалярного произведения (которое может превысить размер кэша) можно раздробить.

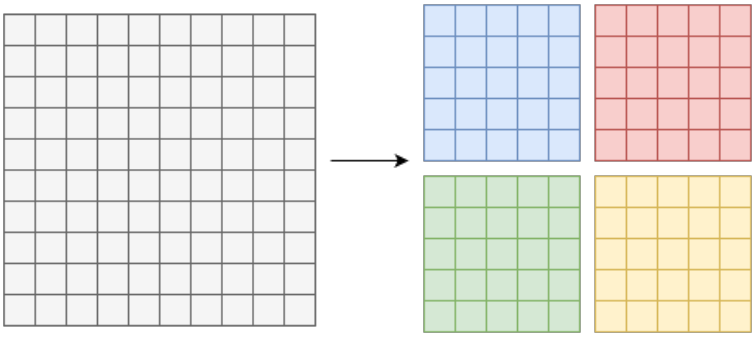

Рис. 4. Tiling

Остановимся подробнее на примере с умножением матриц. Рассмотрим умножение двух матриц размерами 2000 на 2000.

```
%c_init = tensor.empty() : tensor<2000x2000xf32>
%c = linalg.matmul ins(%a, %b : tensor<2000x2000xf32>, tensor<2000x2000xf32>)
outs(%c_init : tensor<2000x2000xf32>) -> tensor<2000x2000xf32>
```

Данная операция раскрывается в три аффинных цикла.

```
affine.for %arg0 = 0 to 2000 {
  affine.for %arg1 = 0 to 2000 {
    affine.for %arg2 = 0 to 2000 {
      %5 = affine.load %reshape[%arg0, %arg2] : memref<2000x2000xf32>
      %6 = affine.load %alloc[%arg2, %arg1] : memref<2000x2000xf32>
      %7 = affine.load %alloc_1[%arg0, %arg1] : memref<2000x2000xf32>
      %8 = arith.mulf %5, %6 : f32
      %9 = arith.addf %7, %8 : f32
      affine.store %9, %alloc_1[%arg0, %arg1] : memref<2000x2000xf32>
    }
  }
}
```

При применении `affine-loop-tile`-оптимизации с параметром `cache-size` цикл разбивается на блоки, причем их размер подбирается так, чтобы объем используемой в каждом отдельном блоке памяти не превосходил `cache-size`. В следующем примере блоки имеют размер 10 на 10 на 10.

```
#map = affine_map<(d0) -> (d0)>
#map2 = affine_map<(d0) -> (d0 + 10)>
affine.for %arg0 = 0 to 2000 step 10 {
  affine.for %arg1 = 0 to 2000 step 10 {
```

```
    affine.for %arg2 = 0 to 2000 step 10 {
      affine.for %arg3 = #map(%arg0) to #map2(%arg0) {
        affine.for %arg4 = #map(%arg1) to #map2(%arg1) {
          affine.for %arg5 = #map(%arg2) to #map2(%arg2) {
            %5 = affine.load %reshape[%arg3, %arg5] : memref<2000x2000xf32>
            %6 = affine.load %alloc[%arg5, %arg4] : memref<2000x2000xf32>
            %7 = affine.load %alloc_1[%arg3, %arg4] : memref<2000x2000xf32>
            %8 = arith.mulf %5, %6 : f32
            %9 = arith.addf %7, %8 : f32
            affine.store %9, %alloc_1[%arg3, %arg4] : memref<2000x2000xf32>
          }
        }
      }
    }
  }
}
```

Исследуем выигрыш от данной оптимизации. Эксперименты проводились на процессоре Apple M1 Pro, обладающем 64 КБ L1-кэша. На графике 5 представлено время выполнения одной операции матричного умножения матриц размера 2000 на 2000, состоящих из четырехбайтовых чисел с плавающей точкой, в зависимости от параметра `cache-size`. Для каждого `cache-size` было проведено 10 измерений, лучший и худший результаты отброшены, оставшиеся усреднены.

Рис. 5. График зависимости времени выполнения операции матричного умножения от значения параметра `cache-size`

7 Заключение

Таким образом, в рамках курсовой работы был реализован прототип компилятора нейронных сетей в формате GGUF, проанализировано пространство для оптимизаций. Перечислим перспективы дальнейших исследований и разработок:

- Поддержка других архитектур больших языковых моделей. Для этого необходимо реализовать дополнительные операции GGML, добавить их в контекст.
- Работа с другими типами. В то время как добавление стандартных типов, таких как `float16` или `float64`, не должно составить труда, интерес представляют квантованные типы, описанные в главе 5.3.2. Для этого необходимо исследовать возможности MLIR по добавлению пользовательских типов.
- Компиляция всего процесса взаимодействия с большой языковой моделью, а не только тензорного вычислительного графа. Для этого необходимо поддержать работу со словарями, токенизацию и прочее.

Библиография

1. Martín~Abadi и др. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems [электронный ресурс]. 2015. URL: https://www.tensorflow.org/.

2. Дмитрий Г. Исходный код проекта [электронный ресурс]. URL: https://github.com/didedoshka/llama.cpp-to-MLIR (дата обращения: 31.03.2025).

3. Georgi Gerganov g.-o. ggml [электронный ресурс]. URL: https://github.com/ggml-org/ggml (дата обращения: 29.03.2025).

4. Ollama Inc. Ollama: Get up and running with large language models locally [электронный ресурс]. 2026. URL: https://ollama.com/ (дата обращения: 30.03.2025).

5. Georgi Gerganov g.-o. gguf [электронный ресурс]. URL: https://github.com/ggml-org/ggml/blob/master/docs/gguf.md (дата обращения: 29.03.2025).

6. Georgi Gerganov g.-o. llama.cpp [электронный ресурс]. URL: https://github.com/ggml-org/llama.cpp (дата обращения: 29.03.2025).

7. Lattner C. и др. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation // 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO). 2021. сс. 2–14.

8. Lattner C., Adve V. LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation. San Jose, CA, USA, 2004. сс. 75–88.

9. Cytron R. и др. Efficiently Computing Static Single Assignment Form and the Control Dependence Graph // ACM Transactions on Programming Languages and Systems. 1991. т. 13, № 4. сс. 451–490.

10. Kennedy K., Allen R. Optimizing Compilers for Modern Architectures: A Dependence-based Approach. San Francisco, CA, USA: Morgan Kaufmann, 2001.